

RFC 0001 — Agent Runtime Control Protocol (ARCP)

Status: Draft (Revision 2)

Authors: Nick Ficano et al.

Protocol Version: 1.0

Abstract

ARCP (Agent Runtime Control Protocol) is a transport-agnostic, schema-first protocol for secure, observable, streaming-native execution of tools, resources, workflows, and agent-to-agent interactions.

ARCP is designed to complement existing capability-discovery protocols such as Model Context Protocol (MCP), while addressing gaps in:

- runtime execution
- streaming
- cancellation
- resumability
- durable jobs
- multi-agent orchestration
- state synchronization
- permissions
- tracing
- event delivery
- sandbox enforcement
- capability negotiation
- authentication and runtime identity
- human-in-the-loop interaction
- artifact addressing
- standardized observability

ARCP is not intended to replace MCP. Instead:

- **MCP** defines *what* exists.
- **ARCP** defines *how* execution occurs.

This revision (v2 draft) tightens contracts that were previously gestural — heartbeats, cancellation, error semantics — and introduces first-class primitives for authentication, human input, subscriptions, artifacts, and protocol extensions.

1. Goals

1.1 Primary Goals

ARCP aims to provide:

- Transport-independent execution semantics
 - Authenticated, attested session establishment
 - Durable asynchronous job execution
 - Streaming-first interactions
 - Typed capability negotiation
 - First-class human-in-the-loop primitives
 - Structured observability and tracing
 - Standardized cost and usage metrics
 - Secure sandboxed execution
 - Agent-to-agent interoperability
 - Backpressure-aware streaming
 - Resumable workflows
 - Unified event propagation
 - Stateful and stateless execution modes
 - Incremental partial responses
 - Addressable artifacts for non-inline outputs
 - Passive observation by third-party clients
 - A defined extension mechanism for incremental evolution
-

2. Non-Goals

ARCP intentionally does **not** define:

- LLM prompt formats
- Vector database standards
- Model architectures
- Tool schema formats
- UI rendering systems
- Authentication provider implementations (the protocol defines exchange shape; not who issues credentials)
- Persistence engine requirements

ARCP **MAY** integrate with these systems.

3. Terminology

Term	Definition
Agent	Autonomous system capable of executing work

Term	Definition
Runtime	Execution environment implementing ARCP
Tool	Executable function/resource
Session	Stateful interaction scope, established only after successful authentication
Stream	Incremental event/data channel
Job	Durable asynchronous execution
Capability	Declared runtime feature
Envelope	Canonical ARCP message container
Transport	Underlying communication layer
Lease	Temporary, time-boxed permission grant scoped to a resource and operation
Subscription	Read-only event feed established by an observer client
Artifact	Addressable, content-typed payload too large or stateful for inline transport
Identity	Verified attestation of a session participant (kind, version, fingerprint, principal)
Heartbeat	Periodic liveness signal emitted by a running job
Extension	Namespaced message type or field outside the core protocol surface
Observer	Client holding only subscriptions; does not issue commands

4. Design Principles

4.1 Transport Agnostic

ARCP **MUST** support:

- stdio
- WebSocket
- HTTP/2
- QUIC
- Unix sockets
- named pipes
- message queues

without changing protocol semantics.

4.2 Streaming Native

Streaming is a first-class primitive.

All invocations **MAY**:

- stream partial results
- emit events

- emit logs
- emit progress
- emit checkpoints
- emit reasoning chunks (typed thought streams)

4.3 Durable Execution

Long-running jobs **MUST** support:

- persistence
- recovery
- resumability
- cancellation
- heartbeats
- scheduled wake-ups

4.4 Typed Contracts

All protocol messages **MUST**:

- validate against schemas
- include explicit versions
- support negotiation

4.5 Event Driven

Everything is modeled as events. Examples:

- invocation started
- progress updated
- partial response
- checkpoint saved
- cancellation requested
- tool completed
- agent transferred
- permission denied
- human input requested
- artifact produced
- usage metric reported

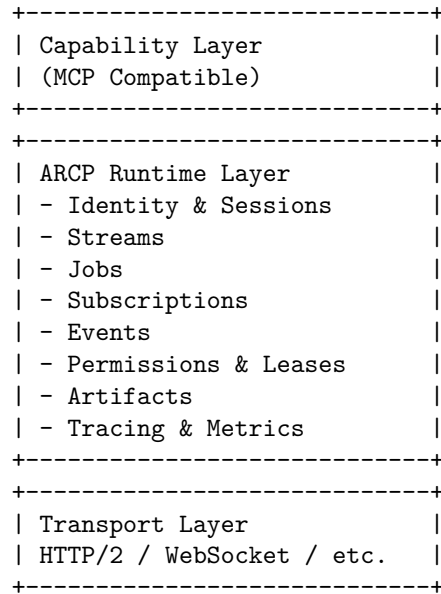
4.6 Authenticated by Default

Sessions **MUST NOT** carry traffic before authentication completes. Anonymous or zero-auth modes are permitted only when explicitly negotiated as a capability and **MUST** be rejected by default.

4.7 Extensible

The protocol **MUST** define a namespaced extension mechanism so vendors and deployments can add message types and fields without forking the core surface, and **MUST** specify how unknown messages are handled.

5. Architecture



Three principal client roles interact with the runtime:

- **Active clients** — issue commands and receive results (e.g. an agent CLI).
 - **Observers** — hold subscriptions only; never command (e.g. a dashboard).
 - **Peer runtimes** — federate via `agent.delegate` and `agent.handoff`.
-

6. Core Protocol Concepts

6.1 Envelope

All ARCP messages **MUST** use a canonical envelope.

Example:

```
{
  "arcp": "1.0",
  "id": "msg_01JABC",
  "type": "job.progress",
}
```

```

"session_id": "sess_123",
"job_id": "job_456",
"trace_id": "trace_789",
"timestamp": "2026-05-07T21:30:00Z",
"idempotency_key": "refund-ord_4812",
"priority": "normal",
"payload": {}
}

```

6.1.1 Envelope Fields

Field	Required	Description
arcp	yes	Protocol version understood by the sender
id	yes	Globally unique message id; transport-level idempotency key
type	yes	Message type, such as <code>tool.invoke</code> , <code>job.progress</code> , <code>stream.chunk</code> , or a namespaced extension type
timestamp	yes	Sender timestamp in RFC 3339 format
source	no	Logical sender id, such as client, runtime, or agent name
target	no	Logical recipient id, such as runtime, tool host, or agent name
session_id	conditional	Required once a session exists
job_id	conditional	Required for durable job events
stream_id	conditional	Required for stream events
subscription_id	conditional	Required for subscription delivery
trace_id	recommended	Stable id for one user-visible request or workflow
span_id	recommended	Span id for the current operation
parent_span_id	optional	Parent span id when the message is part of a trace tree
correlation_id	optional	Id of the command or request this message answers
causation_id	optional	Id of the message that directly caused this message
idempotency_key	optional	Logical idempotency key for the command intent , distinct from <code>id</code> (see §6.4)
priority	no	One of <code>low</code> , <code>normal</code> , <code>high</code> , <code>critical</code> . Default <code>normal</code> . See §6.5
extensions	no	Object of namespaced extension fields (see §21)
payload	yes	Type-specific body validated by the message schema

Receivers **SHOULD** treat message ids as transport idempotency keys. Retrieved messages with the same `id` **MUST NOT** execute twice. Runtimes **SHOULD** preserve `correlation_id` and `causation_id` so clients can reconstruct why an event happened, not only when it happened.

6.2 Message Types

Identity & Authentication

- session.open
- session.challenge
- session.authenticate
- session.accepted
- session.unauthenticated
- session.rejected
- session.refresh
- session.evicted
- session.close

Control

- ping
- pong
- ack
- nack
- cancel
- cancel.accepted
- cancel.refused
- interrupt
- resume
- backpressure
- checkpoint.create
- checkpoint.restore

Execution

- tool.invoke
- tool.result
- tool.error
- job.accepted
- job.started
- job.progress
- job.heartbeat
- job.checkpoint
- job.completed
- job.failed
- job.cancelled
- job.schedule
- workflow.start
- workflow.complete
- agent.delegate
- agent.handoff

Streaming

- stream.open
- stream.chunk
- stream.close

- `stream.error`

Human-in-the-Loop

- `human.input.request`
- `human.input.response`
- `human.choice.request`
- `human.choice.response`
- `human.input.cancelled`

Permissions & Leases

- `permission.request`
- `permission.grant`
- `permission.deny`
- `lease.granted`
- `lease.extended`
- `lease.revoked`
- `lease.refresh`

Subscriptions

- `subscribe`
- `subscribe.accepted`
- `subscribe.event`
- `unsubscribe`
- `subscribe.closed`

Artifacts

- `artifact.put`
- `artifact.fetch`
- `artifact.ref`
- `artifact.release`

Events & Telemetry

- `event.emit`
- `log`
- `metric`
- `trace.span`

Extension messages **MUST** use namespaced types (see §21).

6.3 Command, Result, and Event Flow

ARCP does not require commands to complete synchronously. A command **MAY** be acknowledged immediately, then produce job, stream, log, metric, and trace events over time.

Common flow:

1. Client sends a command, such as `workflow.start` or `tool.invoke`.
2. Runtime returns `ack` or `job.accepted` with `correlation_id` set to the command id.
3. Runtime emits `job.started` when execution begins.
4. Runtime emits `stream.chunk`, `job.progress`, `job.heartbeat`, `log`, `metric`, and `job.checkpoint` events.
5. Runtime emits exactly one terminal event. Direct tool invocations terminate with `tool.result` or `tool.error`. Durable jobs terminate with `job.completed`, `job.failed`, or `job.cancelled`. Workflow-only invocations **MAY** terminate with `workflow.complete`.

If a runtime cannot accept the command, it **MUST** return `nack` or a structured error event with `correlation_id` set to the rejected command id.

6.4 Delivery Semantics

ARCP implementations **SHOULD** support at-least-once delivery for durable jobs. Because messages can be replayed after reconnects, receivers **MUST** deduplicate by `id` and **SHOULD** make tool execution idempotent.

Two distinct keys exist:

- `id` — the *transport* idempotency key. Prevents duplicate execution after retransmits or reconnects.
- `idempotency_key` — the *logical* idempotency key. Prevents the same intent from being executed twice across distinct transport sessions. A client retrying “create refund for order 4812” after a full reconnect **SHOULD** reuse the same `idempotency_key` even though `id` is regenerated.

Runtimes **SHOULD** persist (`session_principal`, `idempotency_key`) for at least the lease horizon of the operation. A repeated logical command **SHOULD** return the previous outcome rather than re-executing.

Ordering is guaranteed only within a `stream_id` or `job_id` unless the transport provides stronger ordering. Clients **SHOULD** use `timestamp`, `correlation_id`, and `causation_id` to rebuild the execution graph.

6.5 Priority and QoS

Senders **MAY** set `priority` on commands, control messages, and event messages. Runtimes **SHOULD**:

- Process higher-priority messages ahead of lower-priority ones within a session, subject to fairness floors that prevent starvation.
- Shed lower-priority traffic first when applying backpressure.
- Never reorder messages within a `stream_id` or `job_id`; priority affects scheduling between streams/jobs, not within them.

`critical` is reserved for messages that must not be deferred (e.g. `permission.request` blocking real human action, terminal job events). Implementations **MAY**

rate-limit **critical** traffic from misbehaving clients.

7. Capability Negotiation

Clients and runtimes **MUST** negotiate capabilities during session establishment.

Example:

```
{
  "capabilities": {
    "streaming": true,
    "durable_jobs": true,
    "checkpoints": true,
    "binary_streams": true,
    "agent_handoff": true,
    "human_input": true,
    "artifacts": true,
    "subscriptions": true,
    "scheduled_jobs": false,
    "extensions": ["arcpv.example.v1", "com.acme.workflow.v2"]
  }
}
```

Either side **MUST** treat absent boolean capabilities as **false**. Required-but-unsupported features **MUST** result in `session.rejected` with code: `UNIMPLEMENTED`.

8. Authentication & Identity

8.1 Session Establishment

Sessions are established with a four-message handshake:

1. Client sends `session.open` with proposed capabilities and an attested client identity block.
2. Runtime responds with either `session.challenge` (if a challenge is required) or `session.accepted` (if the offered credentials suffice).
3. Client responds to challenges with `session.authenticate`.
4. Runtime concludes with `session.accepted` or `session.rejected`.

Until `session.accepted` is received, clients **MUST NOT** send any non-handshake messages. Runtimes **MUST** drop and log other messages received before acceptance.

8.2 Credentials

`session.open` carries a credential block:

```
{
  "type": "session.open",
  "payload": {
    "auth": {
      "scheme": "bearer",
      "token": "..."
    },
    "client": {
      "kind": "claude-code",
      "version": "1.4.2",
      "fingerprint": "sha256:...",
      "principal": "nick@fizzpop.dev"
    },
    "capabilities": { "...": "..." }
  }
}
```

Schemes defined by this revision:

- `bearer` — opaque token; runtime validates against its trust store.
- `mtls` — mutual TLS already established at the transport; payload carries no token, but `fingerprint` **MUST** be present.
- `oauth2` — token is an OAuth 2.0 access token; runtime **MAY** introspect.
- `signed_jwt` — token is a signed JWT with `aud` set to the runtime identity.
- `none` — only valid if `capabilities.anonymous: true` was negotiated; **MUST** be refused otherwise.

8.3 Runtime Identity

Runtimes **SHOULD** include their identity in `session.accepted`:

```
{
  "type": "session.accepted",
  "payload": {
    "session_id": "sess_123",
    "runtime": {
      "kind": "openclaw",
      "version": "0.7.1",
      "fingerprint": "sha256:...",
      "trust_level": "trusted"
    },
    "capabilities": { "...": "..." },
    "lease": { "expires_at": "2026-05-08T03:00:00Z" }
  }
}
```

}

Clients **MAY** pin runtime fingerprints and refuse sessions on mismatch.

8.4 Re-authentication

A runtime **MAY** require re-authentication mid-session by emitting `session.refresh`. The client **MUST** respond with a fresh `session.authenticate` within the deadline. Failure terminates the session with `session.evicted`.

8.5 Eviction

Runtimes **MAY** evict sessions for policy reasons (idle timeout, credential revocation, quota exhaustion). Eviction **MUST** emit `session.evicted` with a reason code drawn from the canonical taxonomy (§18) and **SHOULD** allow in-flight durable jobs to be resumed under a new session.

9. Sessions

Sessions **MAY** be:

- stateless
- stateful
- durable

Stateful sessions **MAY**:

- maintain memory
- preserve auth context
- cache resources
- share execution context across jobs

Durable sessions persist across transport reconnects. Clients reconnect with the same `session_id` and **SHOULD** issue a `resume` message identifying the last observed message id (see §19).

Closing a session via `session.close` is graceful. Open jobs **MUST** be either cancelled, completed, or detached for later resumption according to the runtime's policy and the closer's request.

10. Jobs

10.1 Durable Jobs

Jobs **MUST** support:

- retries

- heartbeats
- checkpoints
- cancellation
- progress reporting

Example:

```
{
  "type": "job.progress",
  "payload": {
    "percent": 42,
    "message": "Embedding documents"
  }
}
```

10.2 Job States

State	Description
accepted	Runtime accepted the command but has not started work
queued	Work is waiting for capacity, permissions, or dependencies
running	Work is actively executing
blocked	Work is waiting on an external event, permission, or human input
paused	Work was intentionally suspended and can be resumed
completed	Work finished successfully
failed	Work reached a terminal error
cancelled	Work was cancelled by a client, runtime, policy, or timeout

Each job **MUST** emit one terminal state. Durable runtimes **SHOULD** persist the last known state, latest checkpoint, retry count, and cancellation reason.

10.3 Heartbeats

Running jobs **MUST** emit `job.heartbeat` at an interval no greater than the negotiated `heartbeat_interval_seconds` (default 30 seconds, advertised in capabilities).

```
{
  "type": "job.heartbeat",
  "job_id": "job_456",
  "payload": {
    "sequence": 17,
    "deadline_ms": 60000,
    "state": "running"
  }
}
```

If a runtime fails to receive N consecutive heartbeats within their declared deadlines (default $N = 2$), it **MUST** transition the job to **failed** with **code**: **HEARTBEAT_LOST**, **OR** to **blocked** if recovery is in progress. Receivers **MUST** advertise their behavior via **capabilities.heartbeat_recovery**: "fail" | "block".

Heartbeats are not progress events. Progress **MUST** be reported separately via **job.progress**.

10.4 Cancellation

cancel is the canonical request to terminate a job, stream, or session.

```
{
  "type": "cancel",
  "payload": {
    "target": "job",
    "target_id": "job_456",
    "reason": "user_aborted",
    "deadline_ms": 5000
  }
}
```

Cancellation is **cooperative**: the runtime **SHOULD** drive the target to a clean checkpoint within **deadline_ms** before terminating. The runtime **MUST** respond:

- **cancel.accepted** — cancellation acknowledged; terminal event will follow within the deadline.
- **cancel.refused** — cancellation rejected (with **reason**, e.g. **not_cancellable**, **already_terminal**).

A successful cancellation **MUST** result in a terminal event:

- **job.cancelled** for jobs
- **stream.error** with **code**: **CANCELLED** for streams
- **session.evicted** with **reason**: **cancelled** for sessions

If **deadline_ms** elapses without progress, the runtime **MAY** escalate to a hard kill and **MUST** emit a terminal event with **code**: **ABORTED**.

10.5 Interrupts

interrupt is distinct from **cancel**: it requests that a running job **pause and accept human guidance**, not terminate.

```
{
  "type": "interrupt",
  "payload": {
    "target": "job",
  }
}
```

```

    "target_id": "job_456",
    "prompt": "Stop and ask before touching production tables."
  }
}

```

The runtime **SHOULD** transition the job to `blocked`, emit a `human.input.request` describing the situation, and resume only after a corresponding response or an explicit `cancel`.

Runtimes that cannot honor interrupts **MUST** advertise `capabilities.interrupt: false`. Clients **MAY** fall back to `cancel`.

10.6 Scheduled Jobs

Clients **MAY** request deferred or recurring execution via `job.schedule`:

```

{
  "type": "job.schedule",
  "payload": {
    "job": { "type": "tool.invoke", "payload": { "...": "..." } },
    "when": { "at": "2026-05-09T13:00:00Z" }
  }
}

```

when **MUST** specify exactly one of:

- `at` — single deferred execution.
- `every` — RFC 5545 RRULE for recurrence.
- `after` — delay relative to acceptance, in seconds.

Runtimes that do not support scheduling **MUST** advertise `capabilities.scheduled_jobs: false` and `nack` schedule requests.

11. Streaming

Streams support:

- text
- binary
- structured events
- logs
- telemetry
- reasoning (“thought”)

Streams **MAY** be multiplexed.

Streams **MUST** support backpressure signaling.

11.1 Stream Kinds

`stream.open` **MUST** declare a kind:

```
{
  "type": "stream.open",
  "stream_id": "str_123",
  "payload": {
    "kind": "thought",
    "content_type": "text/plain",
    "encoding": "utf-8"
  }
}
```

Defined kinds:

Kind	Purpose
<code>text</code>	Plain text output (e.g. tool output, assistant response)
<code>binary</code>	Opaque bytes (see §11.3)
<code>event</code>	Structured JSON events
<code>log</code>	Structured log lines
<code>metric</code>	Telemetry samples
<code>thought</code>	Model reasoning / chain-of-thought (see §11.4)

Receivers **MAY** choose to render, store, or discard streams by kind. Implementations **SHOULD** treat unknown kinds as `event`.

11.2 Backpressure

Clients and runtimes **MAY** send backpressure messages when they cannot process a stream at the current rate.

Example:

```
{
  "type": "backpressure",
  "stream_id": "str_123",
  "payload": {
    "desired_rate_per_second": 20,
    "buffer_remaining_bytes": 65536,
    "reason": "client_render_queue_full"
  }
}
```

Senders **SHOULD** slow or batch `stream.chunk` events after receiving backpressure. Runtimes **SHOULD** shed lower-priority traffic first (§6.5).

11.3 Binary Encoding

For streams of `kind: binary`, two encodings are defined:

- **In-envelope (base64).** `stream.chunk.payload.data` carries base64-encoded bytes. Suitable for small payloads. The envelope **MUST** include `payload.content_type` and **MAY** include `payload.sha256` for integrity.
- **Sidecar frames.** On transports that support multiple frame types (notably WebSocket), the runtime **MAY** send a JSON `stream.chunk` envelope followed by one or more transport-native binary frames keyed to the same `stream_id`. Receivers **MUST** correlate by `stream_id` and `payload.sequence`.

Transports that do not support sidecar frames **MUST** use the in-envelope encoding. Runtimes **MUST** advertise the encoding(s) they support via `capabilities.binary_encoding: ["base64", "sidecar"]`.

11.4 Reasoning Streams

Streams of `kind: thought` carry model reasoning. Chunks **SHOULD** be structured:

```
{
  "type": "stream.chunk",
  "stream_id": "str_thoughts",
  "payload": {
    "role": "assistant_thought",
    "content": "Considering whether to read the config first...",
    "redacted": false
  }
}
```

Producers that redact reasoning for compliance reasons **MUST** still emit chunks with `redacted: true` and an empty or summarized `content` field, so observers can reflect that reasoning occurred without exposing it. Subscribers **MAY** filter `kind: thought` to suppress reasoning client-side.

12. Human-in-the-Loop

ARCP defines first-class primitives for runtime-to-human interaction. These are distinct from `permission.request` (§15.4), which is reserved for capability grants.

12.1 Input Requests

`human.input.request` asks a human for arbitrary structured input. The runtime moves the requesting job to `blocked` until a response arrives or the request

expires.

```
{
  "type": "human.input.request",
  "job_id": "job_456",
  "payload": {
    "prompt": "What branch should I create for this fix?",
    "response_schema": {
      "type": "object",
      "properties": { "branch": { "type": "string", "minLength": 1 } },
      "required": ["branch"]
    },
    "default": { "branch": "fix/auto" },
    "expires_at": "2026-05-09T14:00:00Z"
  }
}
```

The corresponding response:

```
{
  "type": "human.input.response",
  "correlation_id": "<id of the request>",
  "payload": {
    "value": { "branch": "fix/jwt-validation" },
    "responded_by": "ntfy:phone",
    "responded_at": "2026-05-09T13:42:11Z"
  }
}
```

Runtimes **MUST** validate value against response_schema and reject invalid responses with nack and code: INVALID_ARGUMENT.

12.2 Choice Requests

For multi-option pickers, human.choice.request is the typed primitive:

```
{
  "type": "human.choice.request",
  "job_id": "job_456",
  "payload": {
    "prompt": "Three test files failed. How should I proceed?",
    "options": [
      { "id": "fix", "label": "Fix and re-run" },
      { "id": "skip", "label": "Skip and continue" },
      { "id": "abort", "label": "Abort the job" }
    ],
    "expires_at": "2026-05-09T14:00:00Z"
  }
}
```

```
}
```

Response:

```
{
  "type": "human.choice.response",
  "correlation_id": "<id of the request>",
  "payload": {
    "choice_id": "fix",
    "responded_by": "telegram:nick",
    "responded_at": "2026-05-09T13:42:11Z"
  }
}
```

12.3 Provenance and Multi-Channel Resolution

When the same human request is fanned out across multiple destinations (phone, email, dashboard), runtimes **MUST**:

- Resolve on the first valid response and ignore subsequent responses (default), **or** use a different policy negotiated as an extension (e.g. quorum).
- Record `responded_by` so audit trails reflect which channel produced the answer.
- Notify other channels of resolution. The relay or runtime **SHOULD** emit `human.input.cancelled` or equivalent destination-side updates so stale prompts are cleared.

12.4 Expiration

Requests **MUST** carry `expires_at`. Runtimes **MUST** emit a terminal event when the deadline passes:

- If `default` is set, the runtime **MAY** synthesize a `human.input.response` with `responded_by`: `"default"` and proceed.
- Otherwise the runtime **MUST** emit `human.input.cancelled` with `code`: `DEADLINE_EXCEEDED` and either fail the blocking job or escalate per policy.

13. Subscriptions and Observation

Observers (dashboards, audit loggers, debuggers) need read-only event access without participating in execution. ARCP defines a typed subscription mechanism for this.

13.1 Subscribe

```
{
  "type": "subscribe",
```

```

"payload": {
  "filter": {
    "session_id": ["sess_123"],
    "trace_id": ["trace_789"],
    "types": ["job.progress", "job.completed", "log", "metric"],
    "min_priority": "normal"
  },
  "since": { "after_message_id": "msg_01JABC" }
}

```

The runtime responds with `subscribe.accepted` carrying a `subscription_id`. Subsequent events are delivered as `subscribe.event` envelopes containing the original event in `payload.event`. Observers **MUST NOT** receive subscription events for sessions they are not authorized to observe.

13.2 Filtering

Filters **MAY** combine `session_id`, `trace_id`, `job_id`, `stream_id`, `types`, and `min_priority`. All conditions are AND-ed; arrays within a field are OR-ed. Runtimes **MUST** reject filters that would expose unauthorized data with `code: PERMISSION_DENIED`.

13.3 Backfill

`since` requests historical replay before live tail. Implementations **MUST** indicate end-of-backfill with a `subscribe.event` carrying a synthetic `event.emit` of type `subscription.backfill_complete`, so observers know the boundary between historical and live.

13.4 Termination

Either side **MAY** terminate with `unsubscribe`. Runtimes **MAY** terminate subscriptions unilaterally with `subscribe.closed` (e.g. on session eviction, auth expiry, or backpressure overflow) carrying a reason code.

14. Multi-Agent Coordination

ARCP defines optional primitives for:

- agent discovery
- delegation
- handoff
- shared context
- distributed workflows

Example:

```

{
  "type": "agent.delegate",
  "payload": {
    "target": "research-agent",
    "task": "Summarize RFCs",
    "context": {
      "trace_id": "trace_789",
      "shared_memory_ref": "art_mem_001",
      "permissions_inherited": ["filesystem.read"]
    }
  }
}

```

Delegated agents **SHOULD** preserve `trace_id` so distributed traces remain coherent. Handoffs (`agent.handoff`) transfer ownership of a session or job and **MUST** include the receiving runtime's identity for verification.

15. Permissions & Security

15.1 Permission Model

Permissions **MUST** be explicit. Examples:

- `filesystem.read`
- `filesystem.write`
- `network.fetch`
- `email.send`
- `shell.execute`
- `payment.refund.create`

15.2 Sandboxing

Runtimes **SHOULD**:

- isolate execution
- restrict network access
- enforce capability boundaries

15.3 Trust Levels

ARCP defines trust classifications:

Level	Description
untrusted	External/public
constrained	Limited access
trusted	Internal

Level	Description
<code>privileged</code>	System-level

15.4 Permission Challenge Flow

Permissioned operations **SHOULD** use a challenge/response flow:

1. Runtime detects an operation that requires a permission not already covered by the session.
2. Runtime emits `permission.request` and moves the job to `blocked`.
3. Client responds with `permission.grant` or `permission.deny`.
4. Runtime resumes, fails, or delegates according to policy.

Permission grants **SHOULD** be scoped to a specific lease, resource, operation, and expiration time.

```
{
  "type": "permission.request",
  "job_id": "job_refund_123",
  "payload": {
    "permission": "payment.refund.create",
    "resource": "order:ord_4812",
    "operation": "refund",
    "reason": "Issue a customer-approved refund",
    "requested_lease_seconds": 300
  }
}
```

15.5 Lease Lifecycle

A lease is the materialized form of a granted permission. Leases have a defined lifecycle:

- `lease.granted` — emitted by the grantor in response to `permission.grant`. Carries `lease_id`, `permission`, `resource`, `operation`, `expires_at`.
- `lease.refresh` — sent by the holder before expiry to request extension.
- `lease.extended` — emitted on successful extension; carries new `expires_at`.
- `lease.revoked` — emitted by the grantor at any time before natural expiry; carries `reason`.

Holders **MUST** treat operations attempted with a revoked or expired lease as failures with `code: PERMISSION_DENIED`.

15.6 Trust Elevation

Sessions **MAY** request temporary elevation of trust level for specific operations using the permission flow with a synthetic permission `trust.elevate.<level>`

(e.g. `trust.elevate.privileged`). Elevation **MUST** be lease-scoped and audited via `metric` and `log` events.

16. Artifacts

Artifacts are addressable, content-typed payloads referenced by id rather than transported inline. They allow runtimes to produce, exchange, and consume large or binary outputs without burdening the streaming wire.

16.1 Artifact References

An `artifact.ref` is the canonical pointer:

```
{
  "artifact_id": "art_01JABC",
  "uri": "arcp://session/sess_123/artifact/art_01JABC",
  "media_type": "application/json",
  "size": 4194304,
  "sha256": "...",
  "expires_at": "2026-05-10T03:00:00Z"
}
```

References **MAY** appear inside any payload where a large value would otherwise be inlined (e.g. `tool.result.payload.value`, `human.input.response.payload.value`, `agent.delegate.payload.context.shared_memory_ref`).

16.2 Storage and Retrieval

- `artifact.put` — uploads an artifact. Body **MAY** be inline (base64 in `payload.data`) or transferred as binary sidecar frames keyed to the message id.
- `artifact.fetch` — requests an artifact by id. The runtime **MAY** respond inline or with a redirect URI for out-of-band fetch.
- `artifact.release` — signals the holder no longer needs the artifact; runtimes **MAY** garbage-collect.

16.3 Lifecycle

Runtimes **MUST** declare retention policy via `capabilities.artifact_retention`: `{ default_seconds, max_seconds }`. Artifacts past their retention **MUST** be removed; subsequent `artifact.fetch` returns `code: NOT_FOUND`. Long-lived artifacts **SHOULD** be persisted to a backing store rather than retained in-memory.

17. Observability

ARCP includes native observability primitives.

17.1 Tracing

All messages **SHOULD** include:

- `trace_id`
- `span_id`

Compatible with:

- OpenTelemetry
- Datadog
- Honeycomb

Cross-runtime delegation **MUST** propagate `trace_id` so distributed traces remain joined.

17.2 Structured Logs

```
{
  "type": "log",
  "payload": {
    "level": "warn",
    "message": "Retrying tool invocation",
    "attributes": { "attempt": 2, "tool": "filesystem.search" }
  }
}
```

Levels: trace, debug, info, warn, error, critical.

17.3 Metrics

metric events carry a name, value, unit, and dimensions:

```
{
  "type": "metric",
  "payload": {
    "name": "tokens.used",
    "value": 1432,
    "unit": "tokens",
    "dims": { "model": "claude-3.5", "kind": "input" }
  }
}
```

17.3.1 Standard Metric Names To enable interoperable dashboards, ARCP reserves the following metric names. Runtimes producing these concepts

MUST use these names with the indicated units; non-standard variants **MUST** be namespaced.

Name	Unit	Notes
<code>tokens.used</code>	tokens	<code>dims.kind</code> <code>input</code> , <code>output</code> , <code>cache_read</code> , <code>cache_write</code>
<code>cost.usd</code>	usd	Decimal USD with up to 6 fractional digits
<code>gpu.seconds</code>	seconds	Wall-clock GPU time, summed across devices
<code>tool.invocations</code>	count	One per <code>tool.invoke</code>
<code>latency.ms</code>	ms	<code>dims.phase</code> <code>queue</code> , <code>exec</code> , <code>total</code>
<code>bytes.in / bytes.out</code>	bytes	Network transfer at runtime boundary
<code>errors.total</code>	count	<code>dims.code</code> carries the canonical error code

Runtimes **MAY** emit additional metrics under namespaced names (e.g. `arcp.x.acme.cache_hit_ratio`).

18. Error Model

18.1 Error Envelope

Errors **MUST** be structured.

```
{
  "type": "tool.error",
  "payload": {
    "code": "RATE_LIMITED",
    "retryable": true,
    "message": "Upstream rate limit exceeded",
    "details": { "retry_after_seconds": 30 },
    "trace_id": "trace_789"
  }
}
```

Required: `code`, `message`. Optional: `retryable`, `cause` (chained error), `trace_id`.

18.2 Canonical Error Codes

ARCP defines a canonical taxonomy. Implementations **MUST** use these codes when applicable; deployment-specific codes **MUST** be namespaced (e.g. `arcp.x.acme.QUOTA_EXCEEDED`).

Code	Meaning
OK	Not an error; reserved
CANCELLED	Operation cancelled by caller, runtime, or policy
UNKNOWN	Unknown error; avoid in favor of a specific code
INVALID_ARGUMENT	Caller passed a malformed or invalid argument
DEADLINE_EXCEEDED	Operation timed out before completion
NOT_FOUND	Referenced entity does not exist
ALREADY_EXISTS	Entity creation conflicted with existing entity
PERMISSION_DENIED	Caller lacks required permission or lease
RESOURCE_EXHAUSTED	Quota or rate limit hit (<code>RATE_LIMITED</code> is an alias)
FAILED_PRECONDITION	Pre-condition unmet (e.g. job not in cancellable state)
ABORTED	Concurrency conflict or hard termination
OUT_OF_RANGE	Argument out of valid range (subset of <code>INVALID_ARGUMENT</code>)
UNIMPLEMENTED	Feature not supported by this runtime
INTERNAL	Internal runtime error
UNAVAILABLE	Transient unavailability; retry MAY succeed
DATA_LOSS	Unrecoverable data loss or corruption
UNAUTHENTICATED	Missing or invalid credentials
HEARTBEAT_LOST	Job missed required heartbeats (§10.3)
LEASE_EXPIRED	Operation attempted with expired lease (§15.5)
LEASE_REVOKED	Operation attempted with revoked lease
BACKPRESSURE_OVERFLOW	Subscription or stream dropped due to overflow

18.3 Retryability and Backoff

Errors **SHOULD** set `retryable` accurately. Retryable codes by default:

- `RESOURCE_EXHAUSTED`, `UNAVAILABLE`, `DEADLINE_EXCEEDED`, `INTERNAL` (with caution), `ABORTED`

Non-retryable by default:

- `INVALID_ARGUMENT`, `NOT_FOUND`, `ALREADY_EXISTS`, `PERMISSION_DENIED`, `FAILED_PRECONDITION`, `UNIMPLEMENTED`, `UNAUTHENTICATED`, `DATA_LOSS`

`details.retry_after_seconds`, when present, **SHOULD** be honored as a floor for the next attempt.

19. Resumability

ARCP supports:

- checkpoint snapshots
- replay
- recovery

- stream resumption
- subscription resumption

Clients **MAY** reconnect and resume execution.

Resume requests **SHOULD** identify the last message id or checkpoint observed by the client.

```
{
  "type": "resume",
  "session_id": "sess_123",
  "job_id": "job_456",
  "payload": {
    "after_message_id": "msg_01JABC",
    "checkpoint_id": "chk_007",
    "include_open_streams": true
  }
}
```

Runtimes **MUST** be deterministic about replay: a `resume` returns the same canonical message stream as the original session up to the resume point, with possible omission of messages older than the configured retention window. If retention has expired, the runtime **MUST** emit `code: DATA_LOSS` and let the client decide whether to proceed.

20. MCP Compatibility

ARCP **MAY** wrap MCP servers.

Example mapping:

MCP	ARCP
tool schema	capability
tool call	job
resource	stream/resource (delegated to MCP)
prompt	invocation payload

ARCP delegates resource semantics to MCP. Implementations that need first-class resource lifecycle (subscribe, invalidate) **SHOULD** model resources as artifacts (§16) or streams of `kind: event`, rather than introducing parallel resource concepts.

21. Extensions

21.1 Naming

Extension message types and envelope fields **MUST** use one of:

- `arcp.x.<vendor-or-domain>.<name>.v<n>` — community/vendor namespace, recommended.
- Reverse-DNS prefix (e.g. `com.acme.workflow.v2`) — equivalent and acceptable.

The bare `x-` prefix is reserved for **transport-internal** experimental fields and **MUST NOT** appear in long-lived deployments.

21.2 Negotiation

Extensions **MUST** be advertised in `capabilities.extensions` during session establishment. A client requesting an extension that the runtime does not advertise **MUST** receive `nack` with `code: UNIMPLEMENTED`.

21.3 Unknown Message Handling

A receiver encountering an unknown message type **MUST**:

- If the type begins with a recognized core prefix it is required to support: respond with `nack` and `code: UNIMPLEMENTED`.
- If the type is namespaced and not advertised: silently drop **only if** the sender explicitly marked the message `extensions.optional: true`. Otherwise respond with `nack` and `code: UNIMPLEMENTED`.

Receivers **MUST NOT** crash, terminate sessions, or alter unrelated state on encountering unknown types.

21.4 Promotion to Core

Extensions that achieve broad adoption **MAY** be promoted to core in subsequent revisions. Promotion **MUST** preserve wire compatibility for at least one revision: the namespaced form continues to work alongside the new core type.

22. Reference Transports

Mandatory

- WebSocket
- stdio

Recommended

- HTTP/2
- QUIC

Transports **MUST** preserve message body and delivery contract. Transports providing native binary frames (WebSocket, QUIC) **SHOULD** support the sidecar binary stream encoding (§11.3); transports without (stdio) **MUST** use base64 in-envelope.

23. Example Lifecycle

1. Open session (`session.open`)
 2. Authenticate (challenge/response)
 3. Negotiate capabilities
 4. Subscribe (observers, optional)
 5. Invoke tool / start workflow
 6. Open stream(s)
 7. Emit progress, heartbeats, metrics
 8. Request human input or permission as needed
 9. Emit checkpoints
 10. Produce artifacts
 11. Complete job
 12. Persist trace
 13. Close session
-

24. Example Invocation

Authenticated tool invocation with progress, a permission challenge, and artifact output:

```
{
  "type": "session.open",
  "id": "msg_001",
  "timestamp": "2026-05-09T13:00:00Z",
  "payload": {
    "auth": { "scheme": "bearer", "token": "..." },
    "client": {
      "kind": "claude-code",
      "version": "1.4.2",
      "fingerprint": "sha256:..."
    },
    "capabilities": {
      "streaming": true,
      "human_input": true,
      "artifacts": true
    }
  }
}
```

```

}
{
  "type": "tool.invoke",
  "id": "msg_010",
  "session_id": "sess_123",
  "trace_id": "trace_789",
  "idempotency_key": "search-ts-files-2026-05-09",
  "payload": {
    "tool": "filesystem.search",
    "arguments": { "query": "*.ts" }
  }
}
{
  "type": "tool.result",
  "correlation_id": "msg_010",
  "payload": {
    "result_ref": {
      "artifact_id": "art_01JABC",
      "uri": "arcp://session/sess_123/artifact/art_01JABC",
      "media_type": "application/json",
      "size": 92413,
      "sha256": "..."
    }
  }
}
}

```

25. Real-World Examples

Concrete examples are included in:

- docs/real-world-examples.md
- examples/customer-support-refund.jsonl
- examples/local-code-review.jsonl
- examples/data-ingestion-workflow.jsonl
- examples/incident-response.jsonl
- examples/agent-relay-human-input.jsonl (*new in v2*)

These examples show how ARCP behaves in common production settings:

- A support copilot that authenticates, looks up an order, requests a scoped refund permission, and streams customer-visible status.
- A local development agent that reviews code, requests write access via lease, patches files, and streams test output as a `kind: text` stream alongside reasoning as `kind: thought`.

- A durable ingestion workflow that checkpoints progress, handles retryable errors, resumes after failure, and produces artifacts for downstream stages.
- A multi-agent incident workflow that delegates work, preserves shared trace context, and requests approval before rollback.
- An agent relay fanning `human.input.request` to multiple destinations and resolving on the first response.

The examples are intentionally transport-neutral. The same envelopes can move over stdio, WebSocket, HTTP/2, QUIC, or a message queue as long as the transport preserves the message body and delivery contract.

26. Future Work

Items previously considered future and **promoted to core in this revision**: authentication, human-in-the-loop, error taxonomy, heartbeat specification, cancellation contract, subscriptions, artifacts, extension mechanism, standardized cost metrics.

Remaining future-work candidates:

- CRDT-based shared state
- Real-time collaborative agents
- WASM execution sandboxes
- GPU scheduling primitives
- Federated runtime mesh and discovery
- Signed capability manifests
- Economic metering and billing models built on §17.3 metrics
- Agent marketplaces
- Workflow-as-data (formal definition of `workflow.start/workflow.complete` payload schemas, branching, parallelism)
- Resource model promotion (if MCP resources prove insufficient for stateful subscriptions)

27. Why ARCP Exists

Current ecosystems lack a unified runtime protocol for:

- authenticated, attested execution
- durable execution
- orchestration
- structured streams
- secure delegation
- observable agent execution
- standardized human-in-the-loop interaction

ARCP provides:

- execution semantics
- lifecycle management
- runtime interoperability
- a single error and metric vocabulary across implementations

while remaining compatible with:

- MCP
 - JSON-RPC
 - OpenAI tools
 - Anthropic tools
 - future agent ecosystems
-

28. Reference Motto

MCP describes capabilities. **ARCP** operationalizes them.